

Systèmes d'exploitation II

Chapitre 3 : La programmation multi-thread

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

Sommaire

- 1 Implantation des processus Linux
- 2 Notion de thread
- 3 Modèles de threads
- 4 Programmer avec les Pthreads
- 5 Conclusion

Objectifs

- Expliquer la différence entre processus et threads
- Comprendre les modèles d'implantation des threads (M:1, 1:1, M:N)
- Concevoir une application multi-threadée simple
- Programmer avec l'API POSIX Threads (Pthreads)
- Identifier les conditions de compétition et les risques liés au partage de l'espace d'adressage
- Utiliser des mécanismes de synchronisation (mutex, sémaphores)

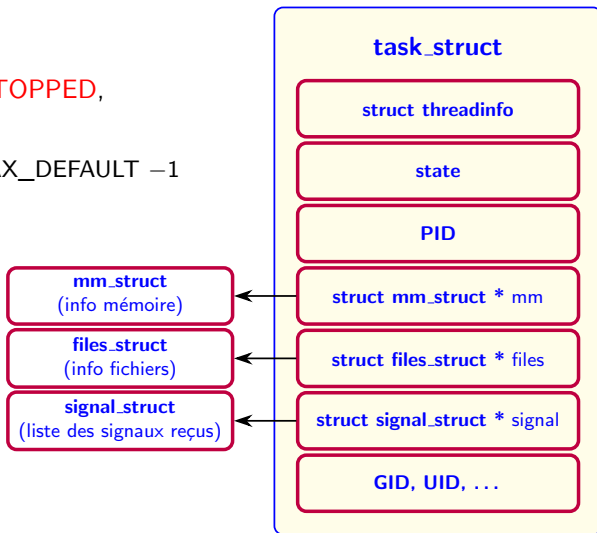
Descripteur de processus (task_struct)

★ Le descripteur de processus (**task_struct**) contient toutes les informations relatives à un processus :

✓ État du processus (state) :

TASK_RUNNING, **TASK_STOPPED**,
TASK_ZOMBIE, ...

✓ pid : 0 .. 32767 = PID_MAX_DEFAULT - 1
/proc/sys/kernel/pid_max



Programme *multi-threadé*

- un programme contenant plusieurs fonctions ou tâches
- ces tâches peuvent être exécutées de manière concurrente, et parfois en parallèle

Un thread (fil d'exécution)

- **En pratique:** une fonction/tâche pouvant s'exécuter indépendamment du main (thread principal)
- **Techniquement:** un **flot d'exécution** pouvant être ordonnancé par l'OS.
Chaque thread ayant son propre contexte d'exécution :

Threads et processus légers sous Linux

- ✓ Un thread = un processus léger (Lightweight Process – LWP).
- ✓ Au sein d'une même application multithreadée :

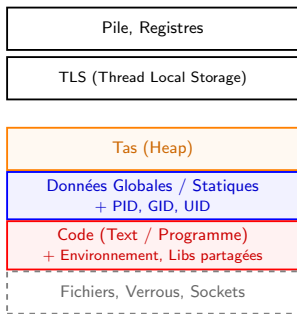
Chaque thread possède en propre :

- Pile
- Registres
- Données spécifiques (TLS - Thread Local Storage)

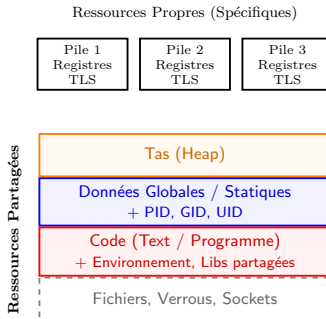
Le reste est partagé :

- ➔ L'ensemble réagit comme un seul processus pour certains appels système.
 - `getpid()` renvoie un pid de groupe `tgid` (thread group leader pid) : pid du premier thread du groupe et non pas pid [POSIX]
 - Si un thread ferme un fichier, **il est fermé pour tous**
 - Si un thread fait `exit()`, **tout s'arrête**
- ➔ Données globales et pointeurs vers le tas: variables partagées (à synchroniser)

Illustration



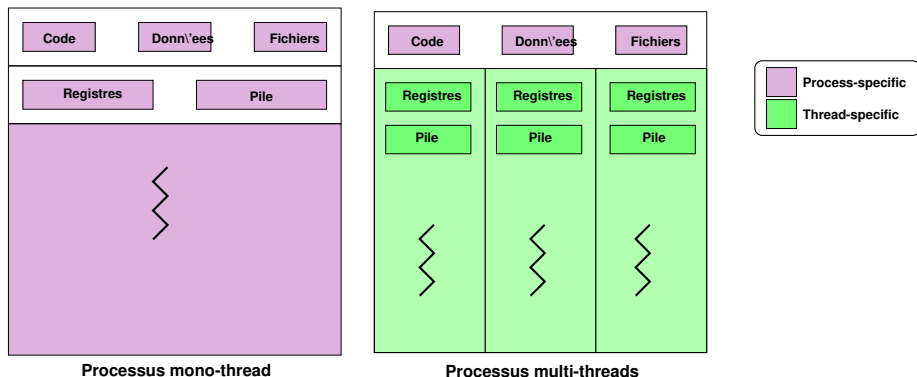
Processus à un seul thread



Processus à plusieurs threads

Processus vs. thread en résumé

- Un processus fournit un espace d'adressage et des ressources (environnement d'exécution) pour les threads
- Les threads sont les unités d'exécution à l'intérieur du processus
- Processus = espace d'adressage + ressources + un ou plusieurs threads



L'espace d'adressage est *passif*; le thread est *actif*

Pourquoi processus dits "légers"?

Les threads contiennent moins de choses

- Partagent la plupart des ressources du processus lourd
- Ne dupliquent que l'indispensable
- $\Rightarrow$ 2 threads dans 1 processus consomment moins de mémoire que 2 processus

Les threads sont plus rapides à créer

- Car il y a moins de choses à dupliquer

Les communications inter-threads sont souvent plus rapides

- les échanges de données passent souvent par la mémoire partagée, sans mécanisme IPC explicite

Usage des threads

Pourquoi utiliser les threads

- **Économie de mémoire** : pas de duplication de l'espace d'adressage.
- **Rapidité** : création et communication plus rapides (pas d'appel système lourds pour échanger des données).
- **Réactivité** : idéal pour les interfaces graphiques ou les serveurs web gérant plusieurs requêtes simultanées.

Quand utiliser les threads

- Recouvrement calcul/communication
- Exécuter des tâches de priorité différentes
- Gérer des événements asynchrones
Tâches indépendantes activées par des événements de fréquence irrégulière
ex. Serveur web peut répondre à plusieurs requêtes en parallèle
- Pour tirer profit des systèmes SMP ou CMP (multi-cœurs)

Quand (pourquoi) ne pas utiliser les threads

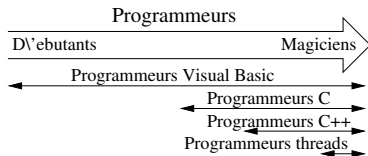
Problèmes du partage de la mémoire

- Besoin d'exclusion mutuelle (condition de compétition)
- Besoin de synchronisation (risque d'interblocage)

⇒ Communication inter-threads rapide mais dangereuse

- Segfault d'un thread → mort de tous
- Casse l'abstraction en modules indépendants
- Extrêmement difficile à debugger (dépendances temporelles; peu d'outils)

Programmer avec les threads,
c'est enlever les gardes-fous de l'OS
pour gagner du temps



(Why Threads are a Bad Idea, USENIX96)

Obtenir de bonnes performances est très difficile

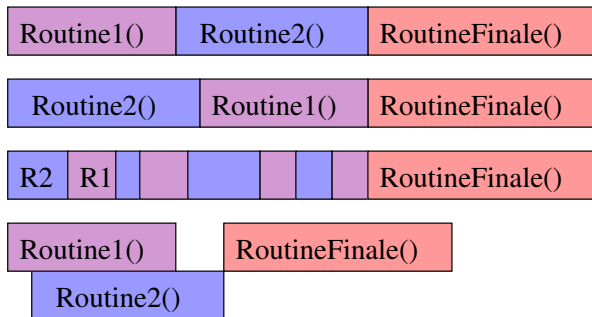
- Verrouillage grossier amène peu de concurrence
- Verrouillage fin augmente la complexité (concurrence pas facilement meilleure)

Design d'applications multi-threadées

Applications candidates

- Applications organisées en tâches indépendantes s'exécutant indépendamment
- Toutes routines pouvant s'exécuter en (pseudo-)parallèle sont candidates
- Interface interactive ($\rightsquigarrow$ latence indésirable) avec des calculs ou du réseau

Dans l'exemple, `routine1()` et `routine2()` sont candidates



Code réentrant et threads-safeness

Code réentrant : Peut être appelé récursivement ou depuis plusieurs contextes d'exécution **sans dépendre** d'un état partagé non protégé

- Ne pas maintenir d'état entre les appels
 - ▶ **Contre-exemple:** strtok, rand (strtok_r est réentrante)
- Ne pas renvoyer un pointeur vers une statique
 - ▶ **Contre-exemple:** ctime (ctime_r est réentrante)

Code thread-safe : peut être utilisé correctement par plusieurs threads simultanément (fonctionne même si utilisé de manière concurrente)

★ Si le code n'est pas réentrant, il faut le protéger par verrous

Problème des dépendances :

- **Votre code est thread-safe, mais vos bibliothèques le sont-elles?**
- La libc est réentrante (linux modernes)
 - ▶ Exemple de errno: chaque thread a maintenant son propre errno (TLS)
- Pour le reste, il faut vérifier (voire, supposer qu'il y a un problème)
- En cas de problème, il faut protéger les appels grâce à un verrou explicite

Patterns classiques avec les threads

Maitre/esclaves

- Un thread centralise le travail à faire et le distribue aux esclaves
- Pool d'esclaves statique ou dynamique
- Exemple: serveur web

Pipeline

- La tâche est découpée en diverses étapes successives
Chaque thread réalise une étape et passe son résultat au suivant
- Exemple: traitement multimédia (streaming)

Peer to peer

- Comme un maitre/esclaves, mais le maitre participe au travail

Implantation des threads: Modèle M:1

Tous les threads d'un processus mappés sur un seul thread noyau
Service implémenté dans une bibliothèque spécifique

- Gestion des threads au niveau utilisateur

- **Avantages:**

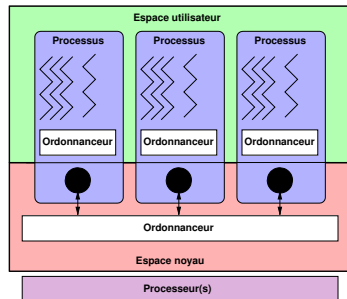
- ▶ Pas besoin de modifier le noyau
- ▶ Rapides pour créer un thread, et pour changer de contexte
- ▶ Portables entre OS

- **Inconvénients:**

- ▶ Ordonnancement limité à celui d'un processus système
⇒ "Erreurs" d'ordonnancement
- ▶ Appel bloquant dans un thread
⇒ blocage de tous
- ▶ Pas de parallélisme
(dommage pour les SMP)

⇒ Efficace, mais sans parallélisme réel

- **Exemples:** Fibres Windows, Java Green Threads, GNU pth (*portable thread*)



(D'après F. Silber)

Modèle 1:1

Chaque thread utilisateur mappé sur un thread noyau

Support noyau des threads

- Gestion des threads au niveau système (LightWeight Process – LWP)

- Avantages

- ▶ Appel bloquant dans un thread
⇒ exécution d'un autre
- ▶ Parallélisme possible
- ▶ Ordonnancement du noyau plus approprié

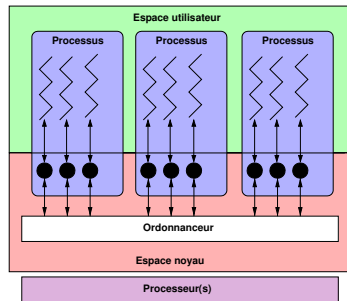
- Inconvénients

- ▶ Besoin d'appels systèmes spécifiques (`clone()` pour la création sous linux)
- ▶ Commutation réalisée par le noyau
⇒ changements de contexte (coûteux)

⇒ Bon niveau de concurrence et vrai parallélisme, au prix d'un coût plus élevé par thread

(nombre total de threads souvent borné)

- Exemples: Windows 95/98/NT/2000, OS/2, Linux, Solaris 9+



(D'après F. Silber)

Modèle M:N

M threads utilisateur mappés sur N threads noyau ($M \geq N \geq 1$)

Services utilisateur basés sur des services noyau

- Coopération entre l'ordonnanceur noyau et un ordonnanceur utilisateur

- **Avantages:** le meilleur des deux mondes

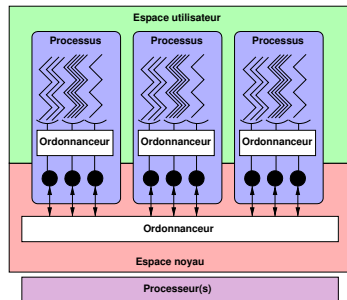
- ▶ Threads noyaux plus basiques et efficaces
- ▶ Threads utilisateurs plus flexibles
- ▶ Moins de changement de contexte
- ▶ Pas d'erreur d'ordonnement

- **Inconvénients:**

- ▶ Extrêmement difficile à implémenter
- ▶ Un peu "usine à gaz":
 - ★ modèle théorique plus pur
 - ★ implémentations complexes
 - ★ efficacité parfois discutable

- Concurrence importante, bonne efficacité (?)

- **Exemples:** Solaris avant v9, IRIX, HP-UX



(D'après F. Silber)

Études de cas (1/3)

Pthreads

- **POSIX threads**: norme standardisée par IEEE POSIX 1003.1c (1995)
- Ensemble de types et fonctions C dont la sémantique est spécifiée
- Il s'agit d'une API : son implémentation dépend du système (linux ou même Windows)

Threads Solaris

- Modèle M:N
- Threads intermédiaires (LWP)

Threads Windows 2000

- Modèle 1:1 pour les WinThreads
- Modèle M:1 pour les fibres (l'OS ne voit pas les fibres)

Threads Java

- Extension de la classe Thread; Implantation de l'interface Runnable
- Implémentation dépend de la JVM utilisée

Études de cas (2/3)

Threads sous linux

- Threads mappés sur des *tâches* (tasks)
- Appel système `clone()` pour dupliquer une tâche (`fork()` implémenté avec ça)

Plusieurs bibliothèques implémentant le standard Pthreads

- Depuis 1996: **LinuxThreads**
 - ▶ Modèle 1:1, par Xavier Leroy (INRIA, créateur d'Ocaml)
 - ▶ Pas complètement POSIX (gestion des signaux, synchronisation)
 - ▶ ≈ 1000 threads au maximum
- **Next Generation POSIX Threads**
 - ▶ Modèle M:N, par IBM; abandonné
 - ↪ NPTL avançait plus vite
 - ↪ M:N induisait une complexité trop importante
- Maintenant **Native POSIX Thread Library**
 - ▶ Modèle 1:1, par Ulrich Drepper (acteur principal libc, chez Red Hat)
 - ▶ totalement conforme à POSIX
 - ▶ bénéficie des évolutions du noyau Linux moderne (meilleure gestion des threads et de l'ordonnancement)
 - ▶ Offre une création de threads plus efficace et plus scalable.

Études de cas (3/3)

Organisation processus / threads dans les OS

	1 espace d'adressage	plusieurs espaces d'adressage
1 thread par processus	Systèmes embarqués simples (sans processus)	Unix historiques processus mono-thread
plusieurs threads par processus	RTOS simples	Linux, Windows, macOS Android, BSD

Généralités sur l'interface Pthreads

Trois grandes catégories de fonctions / types de données

Gestion des threads

- créer des threads, les arrêter, contrôler leurs attributs, etc.

Synchronisation par mutex (**mutual exclusion**)

- créer, détruire, verrouiller, déverrouiller des mutex et contrôler leurs attributs

Variables de condition

- communications entre threads partageant un mutex
- les créer, les détruire, attendre dessus, les signaler; Contrôler leurs attributs

Préfixe d'identificateur	Groupe
pthread_	Threads eux-mêmes et diverses fonctions
pthread_attr_	Attributs des threads
pthread_mutex_	Mutexes
pthread_cond_	Variables de condition

...

...

L'interface Pthreads en pratique

- Types de données sont des structures opaques, fonctions d'accès spécifiques
- L'interface compte 60 fonctions, nous ne verrons pas tout
- Il faut charger le fichier d'en-tête `pthread.h` dans les sources
- Il faut spécifier l'option `-pthread` à gcc

thread-ex1.c

```
#include <pthread.h>
void *hello( void *arg ) {
    int *id = (int*)arg;
    printf("%d: hello world \n", *id);
    pthread_exit(NULL);
}
int main (int argc, char *argv[ ]) {
    pthread_t thread[3];
    int id[3]={1,2,3};
    int i;
    for (i=0;i<3;i++) {
        printf("Thread %d ... \n",i+1);
        pthread_create(&thread[i], NULL,
                      hello, (void *)&id[i]);
    }
    pthread_exit(NULL);
}
```

```
$gcc -pthread -o thread-ex1 thread-ex1.c
$./thread-ex1
Thread 1 ...
Thread 2 ...
1: hello world
Thread 3 ...
2: hello world
3: hello world
$
```

- ★ On ne peut pas savoir dans quel ordre vont démarrer les threads créés
- ✗ Conditions compétition

Identification et création des threads

Identifier les threads

- **pthread_t** : équivalent du pid_t (c'est une structure opaque)
- **pthread_t pthread_self ()** : identité du thread courant
- **int pthread_equal (pthread_t, pthread_t)** : test d'égalité

Créer de nouveaux threads

- Le programme est démarré avec un seul thread, les autres sont créés à la main
- **int pthread_create(identité, attributs, fonction, argument);**
 - ▶ **identité**: [pthread_t*] identifieur unique du nouveau thread (*rempli par l'appel*)
 - ▶ **attributs**: Pour modifier les attributs du thread (NULL → attributs par défaut)
 - ▶ **fonction**: [void (*)(void *)] Fonction C à exécuter (le "main du thread")
 - ▶ **argument**: argument à passer à la fonction. Doit être casté en void*
 - ▶ **retour**: 0 si succès, code d'erreur sinon

Exemple 1 de passage d'arguments

thread-ex2.c

```
#include <pthread.h>
void *hello( void *id ) {
    printf("%d\n", (char *) id);
    pthread_exit(NULL);
}
int main (int argc, char *argv[ ]) {
    pthread_t th[3];
    int i;
    for (i=0;i<3;i++) {
        printf("Thread %d ... \n",i);
        pthread_create(&th[i], NULL, hello,
                      (void *) &i);
    }
    pthread_exit(NULL);
}
```

★ Chaque thread reçoit l'adresse de la variable `i` et la fonction du thread l'affiche

La valeur affichée par chaque thread est-elle garantie ?

→ Non : les threads lisent la même variable partagée, dont la valeur peut avoir changé. ★ C'est une belle condition de compétition.

✓ Passage par valeur `(void *) (intptr_t)i`

✓ Mieux (Passage par référence) comme dans `thread-ex1.c` (slide précédent)

Exemple 2 de passage d'arguments complexes

thread-ex2.c

```
#include <pthread.h>
#define NUM_THREADS 8
typedef struct data {
    int thread_id, sum;
    char *msg;
} data_t;
data_t data_array[NUM_THREADS];
void *PrintHello(void *arg) {
    data_t *mine = (data_t *)arg;
    sleep(1);
    printf("Thread %d:
           %s Sum=%d\n",
           mine->thread_id,
           mine->msg,
           mine->sum);
    pthread_exit(NULL);
}
```

main()

```
pthread_t threads[NUM_THREADS];
int rc, t, sum=0;
char *messages[NUM_THREADS] = {
    "EN: Hello World!",
    "FR: Bonjour, le monde!",
    "SP: Hola al mundo",
    "RU: Zdravstvytye, mir!",
    "DE: Guten Tag, Welt!",
    "Klingon: Nuq neH!",
    "JP: Sekai e konnichiwa!",
    "Latin: Orbis, te saluto!" };
for(t=0; t<NUM_THREADS; t++) {
    sum = sum + t;
    data_array[t].thread_id = t;
    data_array[t].sum = sum;
    data_array[t].msg = messages[t];
    printf("Thread %d ... \n", t);
    rc = pthread_create(&threads[t],
                       NULL, PrintHello,
                       (void*)&data_array[t]);

    if (rc) {
        printf("ERR _create() return %d\n", rc);
        exit(-1);
    }
}
pthread_exit(NULL);
```

Terminaison des threads

Causes de terminaison des threads

- Le thread termine sa fonction initiale
- Le thread appelle la routine `pthread_exit()`
- Le thread est tué par un autre thread appelant `pthread_cancel()`
- Tout le processus se termine à cause d'un `exit`, `exec` ou `return` du `main()`

Terminer le thread courant

- `void pthread_exit(void *retval);`
 - ▶ `retval`: valeur de retour du thread (optionnel)
- Pour récupérer code de retour, un autre thread doit utiliser `pthread_join()`

Attendre la fin d'un thread

Joindre un thread

- `int pthread_join ( pthread_t, void ** )`
- Le thread A joint le thread B: A bloque jusqu'à la fin de B
- Utile pour la synchronisation (rendez-vous)
- Second argument reçoit un pointeur vers la valeur retour du `pthread_exit()`
- Similaire au `wait()` après `fork()`
(mais pas besoin d'être le créateur pour joindre un thread)

Détacher un thread

- `int pthread_detach ( pthread_t )`
- Détacher un thread revient à dire que personne ne le joindra à sa fin
- Le système libère ses ressources au plus vite
- Évite les threads zombies quand on ne veut ni synchro ni code retour
- On ne peut pas ré-attacher un thread détaché

Attributs des threads

Second argument du create

Les fonctions associées

- Création et destruction d'un objet à passer en argument à `pthread_create`:
 - ▶ `int pthread_attr_init ( pthread_attr_t * )`
 - ▶ `int pthread_attr_destroy ( pthread_attr_t * )`
- Lecture : `int pthread_attr_getX ( const pthread_attr_t *, T * )`
- Mise à jour : `int pthread_attr_setX ( pthread_attr_t *, T )`

Attributs existants

- `detachstate` (int): `PTHREAD_CREATE_[JOINABLE|DETACHED]`
- `schedpolicy` (int) : `SCHED_OTHER`, `SCHED_RR`, `SCHED_FIFO`
- `schedparam` (int): priorité d'ordonnancement
- `stackaddr` (void*) et `stacksize` (size_t) pour régler la pile
- `inheritsched`, `scope` (rarement utilisé)

Exemple de gestion des threads

thread-ex3.c

```
#define NUM_THREADS 3
void *travail(void *null) {
    int i;
    double result=0.0;
    for (i=0; i<1000000; i++)
        result = result
            + (double)random();
    printf("Resultat = %e\n",
        result);
    pthread_exit(NULL);
}

int main() {
    pthread_t thread[NUM_THREADS];
    pthread_attr_t attr;
```

```
int rc, t;

/* Init and modify the attribute */
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr,
    PTHREAD_CREATE_JOINABLE);
for(t=0;t<NUM_THREADS;t++) {
    printf("Thread %d ...\n", t);
    if ((rc = pthread_create( &thread[t],
        &attr,
        travail,
        NULL))) {
        printf("ERR _create() : %d\n", rc);
        exit(-1);
    }
}

/* Destroy the attribute */
pthread_attr_destroy(&attr);
/* Wait for other threads */
for(t=0;t<NUM_THREADS;t++) {
    if ((rc = pthread_join(thread[t],
        /*No return value expected*/ NULL))) {
        printf("ERREUR de _join() : %d\n", rc);
        exit(-1);
    }
}
```

Threads et synchronisation

- ✓ Créer des threads est facile
- ✗ Les faire coopérer correctement l'est beaucoup moins notamment lorsqu'il y a partage de ressources :
 - protéger une section critique → mutex
 - coordonner plusieurs threads → sémaphores

Les mutex Pthreads

Interface de gestion

- Type de donnée: `pthread_mutex_t`
- Création:
 - ▶ statique:
`pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;`
 - ▶ dynamique:
`pthread_mutex_init(pthread_mutex_t *, pthread_mutexattr_t *);`
Premier argument: adresse de (pointeur vers) la variable à initialiser
- Destruction: `pthread_mutex_destroy(pthread_mutex_t *)` (doit être déverrouillé)

Interface d'usage

- `pthread_mutex_lock(mutex)` Bloque jusqu'à obtention du verrou
- `pthread_mutex_trylock(mutex)` Obtient le verrou ou renvoie EBUSY
- `pthread_mutex_unlock(mutex)` Libère le verrou

Usage des mutex

Les mutex sont un "gentlemen's agreement"

Thread 1

```
Lock
A = 2
Unlock
```

Thread 2

```
Lock
A = A+1
Unlock
```

Thread 3

```
A = A*B
```

- Thread 3 crée une condition de compétition même si les autres sont disciplinés
(cf. les verrous consultatifs sur les fichiers)
- Pas d'équivalent des verrous impératifs sur la mémoire
- Pas de moniteurs dans PThreads (durs à implémenter en C)

Sémaphores POSIX

→ On parle ici de l'interface POSIX, pas de celle IPC Système V

```
#include <semaphore.h>
```

Type: `sem_t`

Interface de gestion

- `int sem_init(sem_t *sem, int pshared, unsigned int valeur)`
pshared!=0 ⇒ sémaphore partagé entre plusieurs processus
- `int sem_destroy(sem_t * sem)` (personne ne doit être bloqué dessus)

Interface d'usage

- `int sem_wait(sem_t * sem)` réalise un P()
- `int sem_post(sem_t * sem)` réalise un V()
- `int sem_trywait(sem_t * sem)` P() ou renvoie EAGAIN pour éviter blocage
- `int sem_getvalue(sem_t * sem, int * sval)`

Résumé du chapitre

- Définition de threads (vs. processus): *un processus fournit un environnement d'exécution (espace d'adressage, ressources) ; thread=fil d'exécution*
- Avantages et inconvénients: *moins de mémoire, plus rapide/réactif mais c'est plus dangereux*
- Schémas d'implémentations:
 - ▶ Modèle M:1: *M threads par processus (portable et rapide, pas parallèle)*
 - ▶ Modèle 1:1: *Threads gérés par l'OS directement (Linux : un thread = une tâche noyau (LWP))*
 - ▶ Modèle M:N: *Gestion conjointe (modèle théorique meilleur, difficile à faire efficacement)*
- Les fonctions principales de l'interface POSIX:
 - ▶ `pthread_create`: *crée un nouveau thread*
 - ▶ `pthread_exit`: *termine le thread courant*
 - ▶ `pthread_join`: *attend la fin du thread et récupère son errcode*
 - ▶ `pthread_detach`: *indique que personne ne rejoindra ce thread*
 - ▶ `mutex_{lock,unlock,*}`: *usage des mutex*
 - ▶ `sem_{wait,post,*}`: *usage des sémaphore*